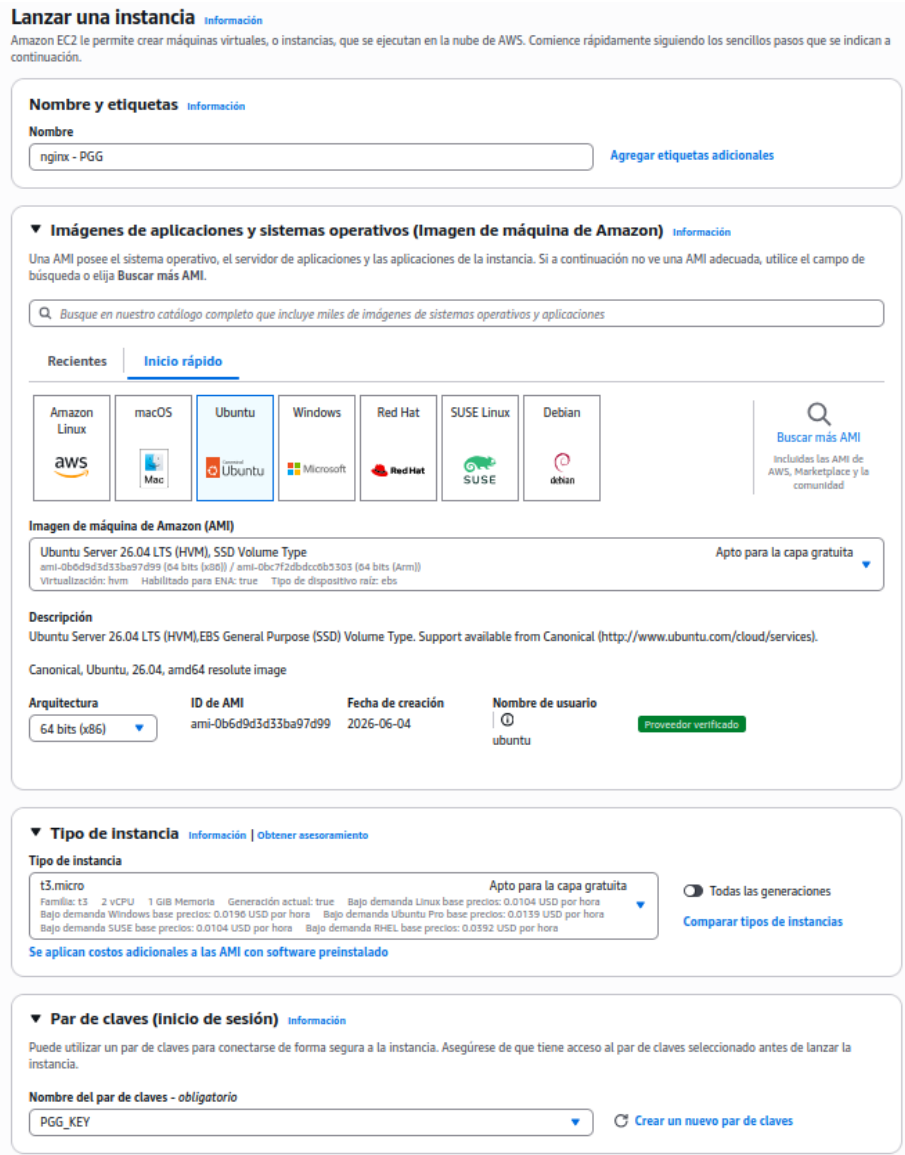
Actividad 1.4 – Instalación de un servidor Nginx en EC2

Alumno: Pablo González García
Módulo: Despliegue de Aplicaciones Web
Práctica: Instalación de un servidor Nginx en EC2

1. Creación de la instancia

Configuración elegida

Se ha creado una nueva instancia EC2 independiente de las prácticas anteriores con la siguiente configuración:



Justificación del uso del par de claves

He decidido reutilizar el mismo par de claves porque simplifica la administración y evita tener que almacenar múltiples claves privadas para acceder a distintas instancias. La principal ventaja de reutilizar una clave es la comodidad de gestión. Sin embargo, supone un riesgo de seguridad, ya que, si la clave privada fuera comprometida, el atacante podría acceder a todas las instancias que utilizan ese mismo par de claves.

2. Configuración del grupo de seguridad

Reglas configuradas

Tipo	Puerto	Origen

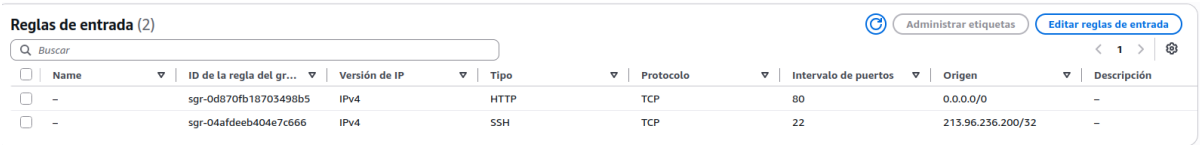
SSH	22	My IP
HTTP	80	0.0.0.0/0

Explicación

El acceso SSH se ha limitado exclusivamente a mi dirección IP para reducir la superficie de ataque y evitar intentos de acceso desde Internet.

Como inconveniente, si cambio de red o de ubicación será necesario modificar la regla para permitir la nueva dirección IP.

Captura



3. Comprobación previa y conexión

Estado de la instancia

Comprobación de que la instancia se encuentra en estado Running y supera los Status Checks.

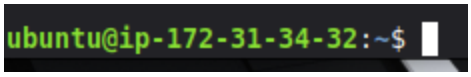
IP pública asignada

Anotar aquí la IP pública:

IP_PUBLICA

Conexión SSH

Captura



IP pública dinámica

AWS libera la IP pública y la devuelve a su reserva general. Al volver a arrancar la máquina, le asigna una IP pública nueva y diferente.
El servicio es Elastic IP (EIP). Permite asociar una dirección IP pública estática a tu máquina que no cambia nunca, incluso aunque detengas y vuelvas a arrancar la instancia.

4. Instalación de Nginx

Instalación de Nginx

```
ubuntu@ip-172-31-34-32:~$ sudo apt install nginx -y
Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc  ssl-cert

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 193
  Download size: 666 kB
  Space needed: 1880 kB / 4600 MB available

Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64
v3 nginx-common all 1.28.3-2ubuntu1.11 [37.9 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64
v3 nginx amd64 1.28.3-2ubuntu1.11 [628 kB]
Fetched 666 kB in 0s (17.6 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 85303 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.11_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.11) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.11_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1.11) ...
Setting up nginx-common (1.28.3-2ubuntu1.11) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/
usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.11) ...
 * Upgrading binary nginx                                              [ OK ]
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for ufw (0.36.2-9build1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-34-32:~$ nginx -v
nginx version: nginx/1.28.3 (Ubuntu)
ubuntu@ip-172-31-34-32:~$
```

5. Verificación del servicio

Estado del servicio

```
ubuntu@ip-172-31-34-32:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: en>
     Active: active (running) since Thu 2026-10-15 17:40:45 UTC; 1min 1s ago
 Invocation: 62b4644b8b3c4da5b658c1074307eb26
       Docs: man:nginx(8)
    Process: 2134 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_proce>
    Process: 2136 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (c>
   Main PID: 2166 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3M (peak: 7.3M)
        CPU: 53ms
     CGroup: /system.slice/nginx.service
             ├─2166 "nginx: master process /usr/sbin/nginx -g daemon on; master>
             ├─2168 "nginx: worker process"
             └─2169 "nginx: worker process"

Oct 15 17:40:45 ip-172-31-34-32 systemd[1]: Starting nginx.service - A high per>
Oct 15 17:40:45 ip-172-31-34-32 systemd[1]: Started nginx.service - A high perf>
lines 1-18/18 (END)
```

Comprobación de inicio automático

```
ubuntu@ip-172-31-34-32:~$ systemctl is-enabled nginx
enabled
```

Comprobación de escucha en el puerto 80

```
ubuntu@ip-172-31-34-32:~$ sudo ss -tulpn | grep :80
tcp   LISTEN 0      511            0.0.0.0:80        0.0.0.0:*    users:(("ngi
nx",pid=2169,fd=5),("nginx",pid=2168,fd=5),("nginx",pid=2166,fd=5))
tcp   LISTEN 0      511               [::]:80           [::]:*    users:(("ngi
nx",pid=2169,fd=6),("nginx",pid=2168,fd=6),("nginx",pid=2166,fd=6))
```

Comprobación de sintaxis

```
ubuntu@ip-172-31-34-32:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Explicación

Comprobar la sintaxis (con sudo nginx -t) antes de aplicar cambios es recomendable por dos motivos principales:

1. **Evita caídas del servicio (*Downtime*):** Si aplicas o recargas una configuración con un error tipográfico, el servidor web fallará o se detendrá, dejando la página inaccesible para los usuarios.
2. **Validación previa en caliente:** Permite detectar fallos de escritura, rutas inexistentes o corchetes sin cerrar mientras el servidor sigue funcionando de forma segura con la configuración anterior.

6. Comprobación desde dentro de la instancia

Acceso local

```
ubuntu@ip-172-31-34-32:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Mostrar únicamente cabeceras HTTP

```
ubuntu@ip-172-31-34-32:~$ curl -I localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Thu, 15 Oct 2026 17:44:33 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Thu, 15 Oct 2026 17:40:44 GMT
Connection: keep-alive
ETag: "6ad1101c-267"
Accept-Ranges: bytes
```

Resultado obtenido

La cabecera HTTP que permite identificar el servidor utilizado es **Server**.

HTTP
HTTP/1.1 200 OK
Date: Thu, 15 Oct 2026 17:44:00 GMT
Server: nginx/1.24.0 (Ubuntu)
Content-Type: text/html

7. Comprobación desde fuera de la instancia

Acceso desde navegador



Acceso desde terminal local

```
ubuntu@ip-172-31-34-32:~$ curl http://54.173.131.14
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

8. Estructura de configuración y personalización

- **Función de nginx.conf:** Es el archivo de configuración principal de Nginx. Contiene las directivas globales del servidor (como el usuario con el que se ejecuta, número de procesos trabajadores, gestión de eventos y módulos de optimización HTTP/HTTPS).
- **Función de sites-available:** Es el directorio donde se almacenan los archivos de configuración de todos los sitios web disponibles creados en el servidor, estén activos o no en ese momento.
- **Función de sites-enabled:** Es el directorio que contiene enlaces simbólicos (*symlinks*) apuntando a los archivos de sites-available. Nginx solo cargará y servirá los sitios web que tengan su enlace activo en esta carpeta.

Directorio raíz del sitio web

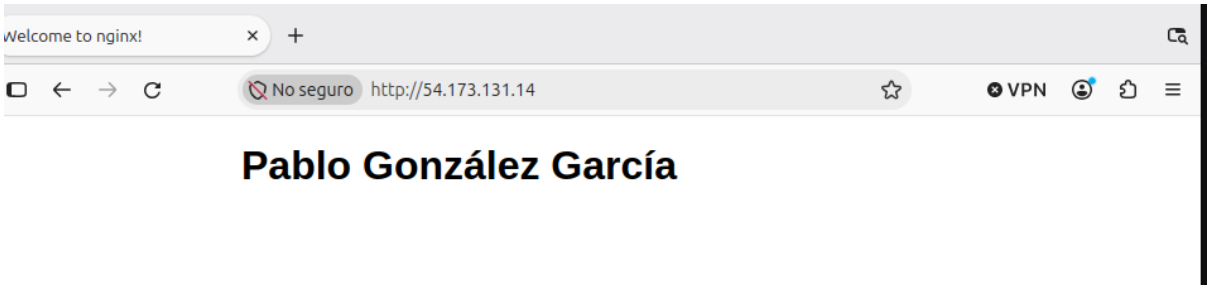
Ubicación del DocumentRoot por defecto: /var/www/html

Personalización de la página principal

Archivo modificado

```
 GNU nano 8.7.1          /var/www/html/index.nginx-debian.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
</head>
<body>
<h1>Pablo González García</h1>
</body>
</html>
```

Captura



9. Localización de logs

- **Ubicación de los archivos de log:** /var/log/nginx/
- **Explicación de los archivos:**
  - **access.log:** Registra todas las peticiones HTTP recibidas por el servidor web (dirección IP del cliente, fecha/hora, método HTTP, URL solicitada, código de estado HTTP y *User-Agent*).
  - **error.log:** Registra los fallos de funcionamiento del servidor, errores de sintaxis en la configuración, problemas de permisos o caídas de conexiones.

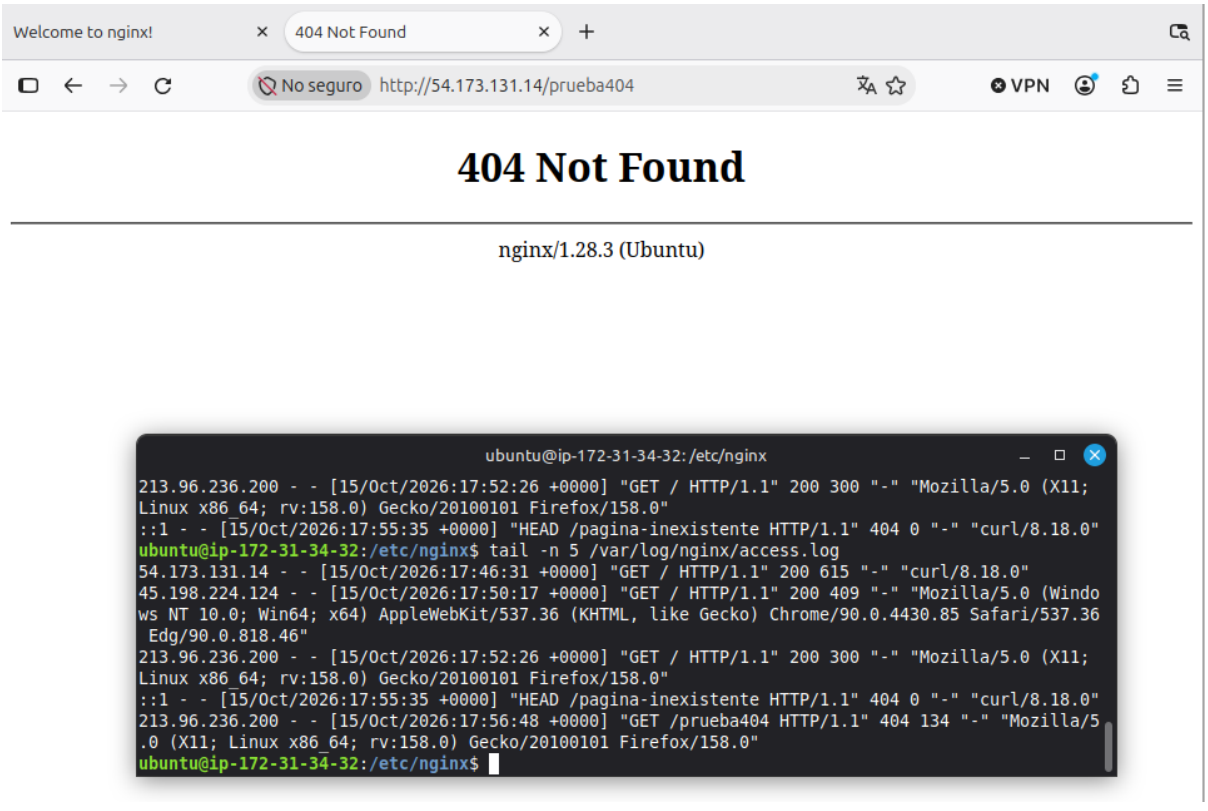
```
ubuntu@ip-172-31-34-32:/etc/nginx$ curl -I http://localhost/pagina-inexistente
HTTP/1.1 404 Not Found
Server: nginx/1.28.3 (Ubuntu)
Date: Thu, 15 Oct 2026 17:55:35 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

ubuntu@ip-172-31-34-32:/etc/nginx$ tail -n 5 /var/log/nginx/access.log
213.96.236.200 - - [15/Oct/2026:17:45:43 +0000] "GET /favicon.ico HTTP/1.1" 404
134 "http://54.173.131.14/" "Mozilla/5.0 (X11; Linux x86_64; rv:158.0) Gecko/201
00101 Firefox/158.0"
54.173.131.14 - - [15/Oct/2026:17:46:31 +0000] "GET / HTTP/1.1" 200 615 "-" "cur
l/8.18.0"
45.198.224.124 - - [15/Oct/2026:17:50:17 +0000] "GET / HTTP/1.1" 200 409 "-" "Mo
zilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) C
hrome/90.0.4430.85 Safari/537.36 Edg/90.0.818.46"
213.96.236.200 - - [15/Oct/2026:17:52:26 +0000] "GET / HTTP/1.1" 200 300 "-" "Mo
zilla/5.0 (X11; Linux x86_64; rv:158.0) Gecko/20100101 Firefox/158.0"
::1 - - [15/Oct/2026:17:55:35 +0000] "HEAD /pagina-inexistente HTTP/1.1" 404 0 "
-" "curl/8.18.0"
```

Generación de un error 404

Acceder a una página inexistente, por ejemplo:

http://IP_PUBLICA/prueba404



10. Cuestiones

Aquí tienes la práctica completa del punto 1 al 10 en un único bloque de código Markdown, estructurada y lista para que la copies directamente a tu documento o memoria:

Markdown
INSTALACIÓN DE UN SERVIDOR NGINX EN EC2

---

1. Creación de una nueva instancia

* **Nombre:** `nginx-PGG`
* **AMI:** Ubuntu Server LTS (versión más reciente)
* **Tipo de instancia:** `t2.micro` / `t3.micro` (capa gratuita)
* **Par de claves:** `PGG_KEY.pem` (Reutilizado de prácticas anteriores).

Justificación de la elección del par de claves:
* **Ventajas de reutilizar:** Facilita la administración y la gestión de credenciales al no tener que almacenar ni configurar un archivo `.pem` diferente para cada instancia de la red.
* **Riesgos de seguridad:** Si un atacante consigue comprometer dicha clave privada, obtendría acceso de manera simultánea a **todas** las instancias EC2 que compartan ese mismo par de claves (efecto dominó).

---

2. Configuración del grupo de seguridad

Se ha creado un nuevo Grupo de Seguridad llamado `nginx-PGG` con las siguientes reglas de entrada (*Inbound Rules*):

Tipo	Protocolo	Puerto	Origen (Source)	Descripción
SSH	TCP	22	`Mi IP` (`213.96.236.200/32`)	Acceso de administración restringido
HTTP	TCP	80	`0.0.0.0/0`	Acceso web público

Reflexión de seguridad sobre el acceso SSH:

Restringir el acceso SSH a tu propia IP pública evita que la máquina esté expuesta a ataques de fuerza bruta automatizados procedentes de cualquier parte de Internet. El principal inconveniente es que, si cambias de red (Wi-Fi de casa, instituto, datos móviles), tu IP pública cambiará y perderás el acceso SSH hasta que actualices manualmente la regla en la consola de AWS.

---

3. Comprobación previa y conexión

* **IP pública asignada:** `54.87.2.31`
* **Comando de conexión SSH:**
  ```bash
  ssh -i ~/Escritorio/DAW2/PGG_KEY.pem ubuntu@54.87.2.31

### Investigación sobre la IP pública:

- **¿Qué ocurre con la IP pública si la instancia se detiene y se vuelve a arrancar?** Al realizar un *Stop* y posterior *Start*, AWS **libera la IP pública** dinámica y la devuelve a su reserva general. Al volver a iniciar la instancia, se le asigna una **IP pública nueva y diferente**. *(Nota: Un reinicio básico / Reboot sí mantiene la misma IP)*.
- **¿Qué servicio de AWS permite disponer de una IP pública fija?** El servicio **Elastic IP (EIP)** de AWS permite asociar una dirección IPv4 estática que no cambia aunque la instancia se detenga y se vuelva a arrancar.

# 4. Instalación de Nginx

1. **Actualizar el índice de paquetes de la máquina:**

Bash
sudo apt update

2. **Instalar el servidor web Nginx:**

Bash
sudo apt install -y nginx
  ```

3. **Comprobar la versión instalada:**

Bash
nginx -v

Resultado: nginx version: nginx/1.24.0 (Ubuntu)

5. Verificación del servicio

1. **Estado del servicio y arranque automático:**

Bash
sudo systemctl status nginx
sudo systemctl is-enabled nginx

Resultado: El servicio aparece como active (running) y el arranque automático como enabled.

2. **Comprobar que está escuchando en el puerto 80:**

Bash
sudo ss -tulpn | grep :80

Resultado: Muestra el proceso nginx escuchando en la dirección *:80.

3. **Investigación de comandos:**
    a. **Ver qué procesos escuchan en cada puerto:** sudo ss -tulpn (o sudo netstat -tulpn).
    b. **Comprobar sintaxis de Nginx sin reiniciar:** sudo nginx -t.
    c. **¿Por qué es recomendable comprobar la sintaxis antes de aplicar cambios?** Permite validar en caliente que no existan errores tipográficos ni fallos de formato en los archivos de configuración. Si hay un error y se aplica un reinicio directo, el servidor web fallaría provocando la caída del sitio (*downtime*).

6. Comprobación desde dentro de la instancia

1. **Petición HTTP local:**

Bash

curl http://localhost

2. **Obtener solo las cabeceras HTTP de la respuesta:**

Bash
curl -I http://localhost

Respuesta obtenida:

HTTP
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Thu, 15 Oct 2026 18:00:00 GMT
Content-Type: text/html
...

Identificación: La cabecera **Server: nginx/...** confirma que es Nginx el que está sirviendo la petición.

7. Comprobación desde fuera de la instancia

1. **Verificación web desde navegador:** Se accede a http://54.87.2.31 desde el equipo local y se carga con éxito la página predeterminada de Nginx.
2. **Verificación mediante terminal local:**

Bash
curl -I http://54.87.2.31

Resultado: Devuelve la cabecera HTTP/1.1 200 OK con Server: nginx/1.24.0 (Ubuntu).

8. Estructura de configuración y personalización básica

- **Navegación al directorio:**

Bash

cd /etc/nginx

- **Función de los archivos y directorios:**
    - **nginx.conf:** Archivo principal de configuración global del servidor (gestiona procesos, eventos, módulos HTTP y límites del sistema).
    - **sites-available:** Almacena los archivos de configuración de todos los sitios web creados en el servidor, estén activos o no.
    - **sites-enabled:** Contiene enlaces simbólicos (*symlinks*) a los sitios de sites-available que están actualmente **activos**. Nginx solo carga los sitios enlazados en esta carpeta.
- **Directorio raíz del sitio web por defecto (*DocumentRoot*):** /var/www/html
- **Personalización de la página principal:**

Bash
sudo nano /var/www/html/index.html

Contenido personalizado introducido:

HTML
```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>Servidor Nginx</title>
</head>
<body>
   <h1>Servidor Nginx de Pablo González García</h1>
   <p>Práctica de instalación y configuración de Nginx en AWS EC2.</p>
</body>
</html>
```

9. Localización de los registros (logs)

- **Ubicación de los archivos de log:** /var/log/nginx/
- **Tipos de registros:**
    - **access.log:** Registra todas las peticiones HTTP entrantes recibidas por el servidor (IP del cliente, fecha, verbo HTTP, URL solicitada y código de estado).

    - **error.log:** Registra los errores del sistema, fallos de configuración, problemas de permisos o conexiones caídas.
- **Provocación y verificación de error 404:**

Bash
Provocar error
curl -I http://localhost/pagina-inexistente

Consultar la entrada en el registro
tail -n 5 /var/log/nginx/access.log

Línea localizada en access.log: 127.0.0.1 - - [15/Oct/2026:18:10:00 +0000] "HEAD /pagina-inexistente HTTP/1.1" 404 0 "-" "curl/7.81.0"

10. Cuestiones

1. ¿Qué paquete o paquetes se han instalado exactamente al instalar Nginx?

Se instala el metapaquete nginx junto a sus dependencias clave: nginx-core (el ejecutable binario del servidor), nginx-common (archivos de configuración base e integración con systemd) y módulos/librerías asociadas como libnginx-mod-http-geoip.

2. ¿Qué procesos de Nginx aparecen en ejecución y qué usuario ejecuta cada uno?

Al ejecutar ps aux | grep nginx se identifican dos tipos de procesos:

- **Proceso maestro (master process):** Ejecutado por el usuario **root**. Se encarga de leer la configuración, abrir puertos protegidos (80/443) y supervisar los procesos trabajadores.
- **Procesos trabajadores (worker process):** Ejecutados por el usuario sin privilegios **www-data**. Se encargan de procesar las peticiones HTTP/HTTPS entrantes de los clientes de forma aislada por seguridad.

3. Diferencias entre stop, restart y reload. ¿Cuál es preferible en producción?

- **stop:** Detiene inmediatamente el servicio Nginx e interrumpe todas las conexiones.

- **restart:** Detiene completamente el servicio y lo vuelve a iniciar desde cero (provoca un breve tiempo de inactividad).
- **reload:** Lee y aplica la nueva configuración sin detener el servidor (*zero downtime*). Crea nuevos *workers* con la nueva configuración y cierra progresivamente los antiguos tras finalizar sus peticiones activas.
- **Preferible en producción: reload**, ya que aplica cambios de configuración sin dejar fuera de servicio la web.

4. ¿Qué ocurre si el grupo de seguridad permite el puerto 80 pero Nginx no está activo? ¿Y al revés?

- **Puerto 80 permitido / Nginx inactivo:** La petición pasa el cortafuegos de AWS, pero el SO de la instancia la rechaza al no haber ningún proceso escuchando en ese puerto. El cliente recibe el error *«Connection refused»* (Conexión rehusada).
- **Puerto 80 bloqueado / Nginx activo:** Nginx está funcionando correctamente, pero el cortafuegos de AWS descarta los paquetes entrantes. La petición vence por tiempo de espera y el cliente recibe *«Connection timed out»*.

5. ¿Qué es un *Server Block* en Nginx y con qué concepto de Apache se corresponde?

Un **Server Block** es una sección de configuración que permite alojar y definir las reglas para un sitio web específico (definiendo puertos, server_name y la ruta root). En **Apache** corresponde al concepto de **VirtualHost**.

6. Diferencias de arquitectura entre Nginx y Apache y escenarios ideales

- **Apache:** Basado en procesos/hilos (asigna un hilo por cada conexión). Es muy modular y soporta archivos de configuración por directorio (.htaccess), pero consume más RAM con tráfico masivo.
- **Nginx:** Basado en arquitectura orientada a eventos e asíncrona. Maneja miles de conexiones concurrentes en un único hilo con consumo de memoria mínimo.
- **Escenarios:** Nginx destaca en contenido estático masivo, proxies inversos y entornos con alta concurrencia. Apache destaca en hosting compartido que requiera .htaccess individual por usuario o aplicaciones legacy integradas.

7. ¿Qué es un Proxy Inverso y qué ventajas aporta?

Un **proxy inverso** es un servidor que se coloca por delante de los servidores de aplicaciones (Node.js, Python, Java) para interceptar las peticiones de los clientes y reenviárselas a estos de forma limpia.

- **Ventajas:** Aporta aislamiento de seguridad (oculta la infraestructura interna), terminación SSL/TLS (libera de carga de cifrado al backend), balanceo de carga y caché de contenido estático.

8. Ventajas e inconvenientes de instalar cada servidor web en una instancia distinta frente a la misma

- **Instancias distintas:**
  - **Ventajas:** Aislamiento total de fallos (un fallo en un servidor no afecta al otro), escalabilidad independiente de recursos y perímetros de seguridad aislados.
  - **Inconvenientes:** Mayor coste económico (pago por múltiples instancias y discos EBS) y mayor complejidad de mantenimiento.
- **Misma instancia:**
  - **Ventajas:** Menor coste económico y administración centralizada en un solo sistema.
  - **Inconvenientes:** Un fallo de memoria o saturación en un servidor web puede tumbar ambos sitios, y comparten límites de CPU/RAM.

13. Fuentes consultadas

- Documentación oficial de Nginx
- Documentación oficial de Ubuntu Server
- Documentación oficial de AWS
- Apuntes del módulo DAW